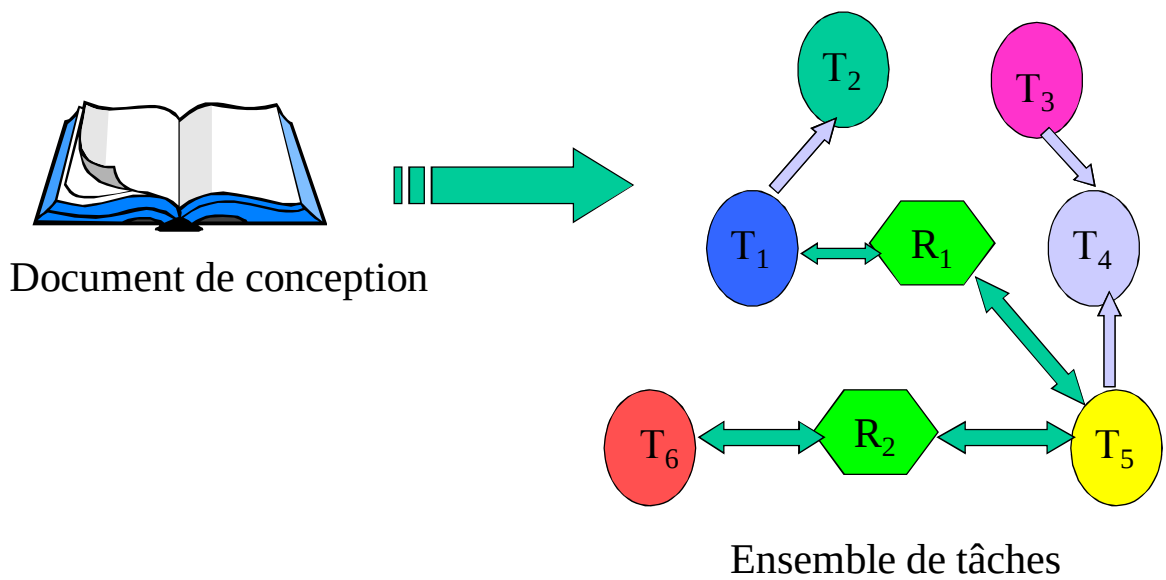


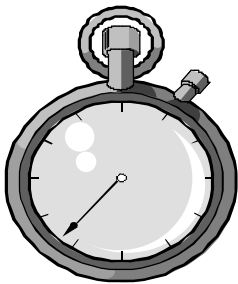
Chapitre 2

Généralités sur l'ordonnancement de tâches

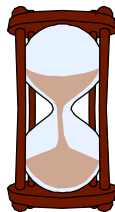
1. Modèles de tâches (contraintes)



Contraintes de tâches (1/5)



Top départ



Durée d'exécution



Echéance

→ individuelles

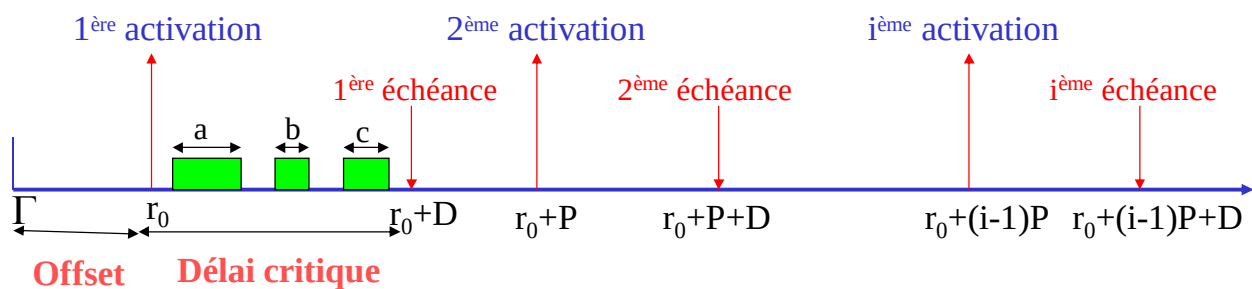
→ collectives



Contraintes de tâches (2/5)

Modèle simple (tâches périodiques)

Mesure de température



Γ : instant initial (démarrage de l'application)

r_0 : date de première activation

C : pire durée d'exécution = $\text{Max}(a) + \text{Max}(b) + \text{Max}(c)$

D : délai critique (Durée d'exécution max, démarre au début de la tâche mais pas à l'activation)

P : période Si $P = D$: **échéance sur requête**

Tâche = (r_i, C_i, D_i, P_i)

r_i = release time

C_i = Computation time

D_i = Deadlim

P_i = Period

Charge-processeur d'une application TR

$$u_i = \frac{C_i}{P_i} \quad : \text{ Pourcentage de l'activité du processeur dédiée à la tâche } T_i$$

$$U = \sum_{i=1}^n \frac{C_i}{P_i} \quad : \text{ Charge processeur = taux d'utilisation du processeur pour les tâches périodiques}$$

$U > m \Rightarrow$ application non ordonnançable sur m processeurs

$(U \leq m$: Condition nécessaire, mais pas suffisante)

Pas suffisant car :

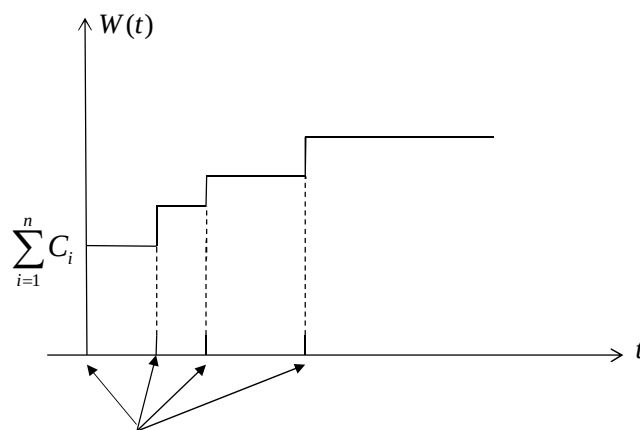
T1 : $P_i = 10, C_i = 5$ |-----
 T2 : $P_i = 4, C_i = 2$ |--_____
 NON ORDONNANCABLE !

Demande de processeur

Quantité de travail totale demandée au processeur à l'instant t : $W(t)$

$$W(t) = \sum_{i=1}^n \left(1 + \left\lfloor \frac{t}{P_i} \right\rfloor \right) \times C_i$$

Total du travail demandé au processeur depuis le début de l'exécution



Instants de demande d'activation de tâche

Plus longue période d'activité du processeur

Cas de tâches périodiques

Pire cas = cas où toutes les tâches sont activables à l'instant $t=0$

Plus longue période d'activité du processeur L est donnée par :

$$L = \sum_{i=1}^n \left\lceil \frac{L}{P_i} \right\rceil \times C_i$$

Point fixe $\Rightarrow$ calcul itératif

$$\begin{cases} L^{m+1} = W(L^m) \\ L^0 = \sum_{i=1}^n C_i \end{cases} \quad \text{On s'arrête quand } L^{m+1} = L^m$$

Plus longue période d'activité du processeur

Cas de tâches périodiques

Exemple de calcul de L

	C_i	P_i
T_1	7	20
T_2	5	20
T_3	8	30
T_4	3	100
T_5	2	100

$$L^0 = \sum_{i=1}^n C_i = 7 + 5 + 8 + 3 + 2 = 25$$

$$L^1 = W(L^0) = \sum_{i=1}^n \left(1 + \left\lfloor \frac{25}{P_i} \right\rfloor \right) \times C_i =$$

$$(2 \times 7) + (2 \times 5) + (1 \times 8) + (1 \times 3) + (1 \times 2) = 37$$

$$L^2 = W(L^1) = \sum_{i=1}^n \left(1 + \left\lfloor \frac{37}{P_i} \right\rfloor \right) \times C_i = 45$$

$$L^3 = W(L^2) = \sum_{i=1}^n \left(1 + \left\lfloor \frac{45}{P_i} \right\rfloor \right) \times C_i = 57$$

$$L^4 = W(L^3) = \sum_{i=1}^n \left(1 + \left\lfloor \frac{57}{P_i} \right\rfloor \right) \times C_i = 57$$

Contraintes de tâches (3/5)

Modèle “complet” de tâche (contraintes individuelles)

- Type de tâche (**critique** ou **non**)
- Importance de tâche et Priorité de tâche
- Autorisation de réquisition du processeur
- Contraintes temporelles
 - Tâche **périodique**, **sporadique** ou **apériodique**
 - Période (**P_i**)
 - Instant d'arrivée (**r_i**)
 - Fixe = r_i
 - Dans un intervalle $[r_i^{\min}, r_i^{\max}]$
 - Cyclique = $K \cdot P + r_0$
 - Avec **Offset** (fixe ou dynamique)
 - Aléatoire
 - sporadique (**Intervalle d'interarrivée minimal connu**)

Contraintes de tâches (4/5)

Modèle “complet” de tâche (contraintes individuelles)

→ Durée d'exécution (C_i)

Maximale (Worst Case Execution Time)

Dans un intervalle $[C_i^{\min}, C_i^{\max}]$

Moyenne, probabiliste, etc.

→ Echéance absolue (date au plus tard de terminaison) (d_i)

→ Echéance relative (temps de réponse maximum) (D_i)

→ Terminaison (totale, partielle, ...)

→ Gigue d'activation, de terminaison... (délais au début et à la fin)

→ Autres

● Contraintes liées à la précision des résultats des tâches

Terminaison partielle : peut être interrompue n'importe quand même plus on attend, mieux c'est (exemple : calcul de π)

Contraintes de tâches (5/5)

Modèle “complet” de tâche (contraintes collectives)

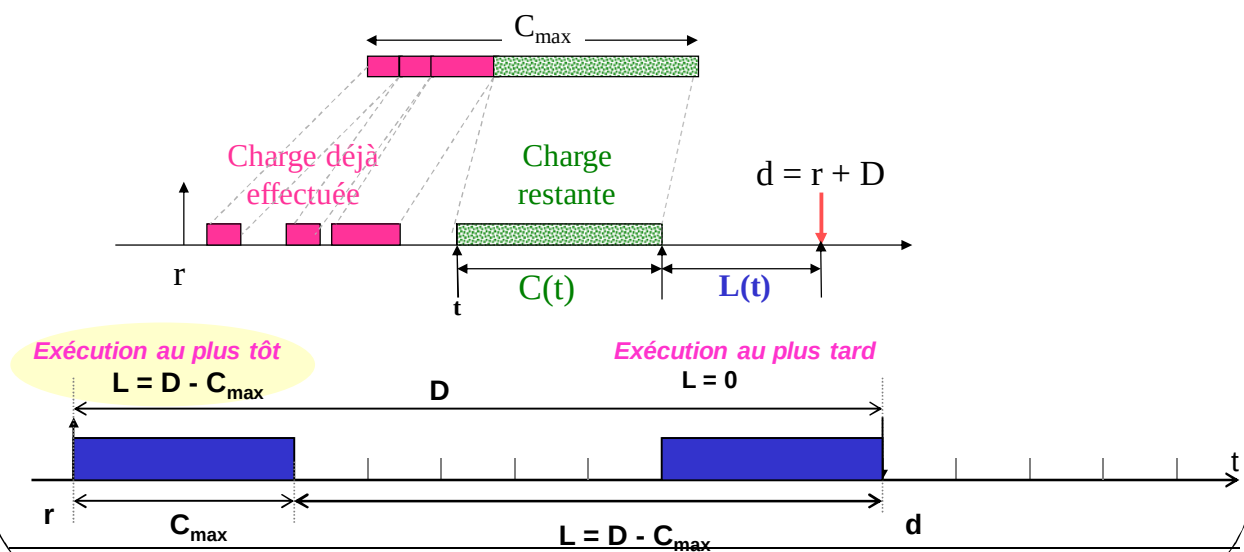
- **Contraintes de relations entre tâches**
 - **Précédence**
 - **Partage de ressources**
- **Contraintes de répartition**
- **Contraintes de tolérance aux fautes**

Paramètres dynamiques de tâche (1/4)

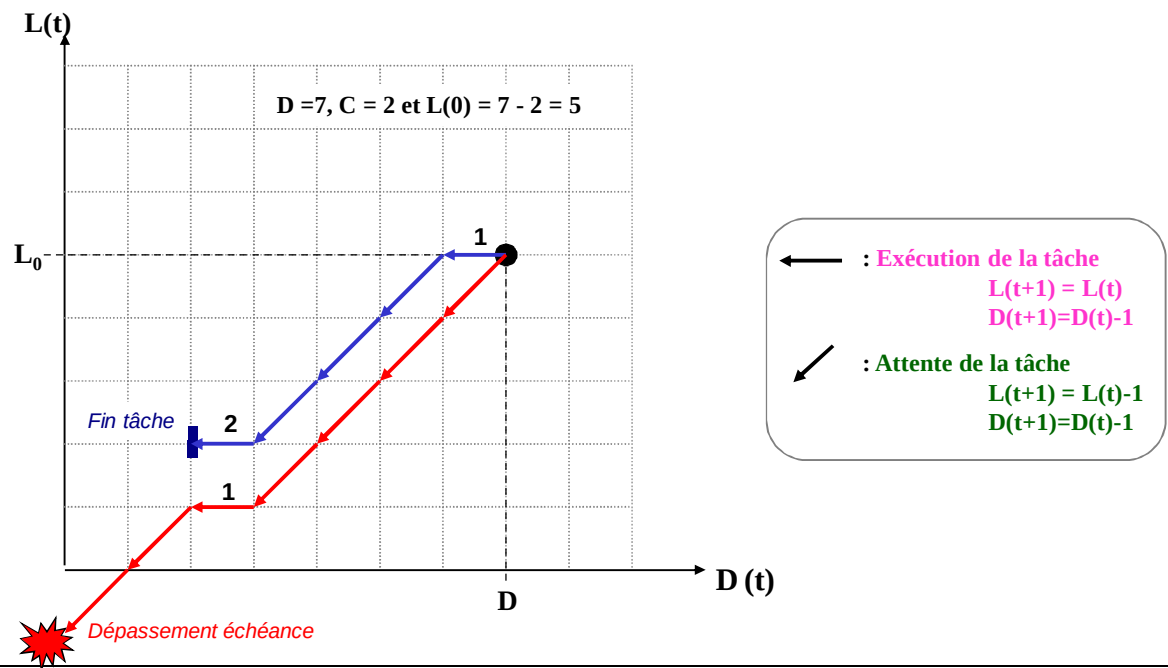
→ temps d'exécution restant $C(t)$: $C(t) = C_{\max} - C_{\text{consommé}}(t)$

→ laxité dynamique $L(t)$: temps avant le début d'exécution sans interruption de la tâche

$$L(t) = d - t - C(t) = r + D - t - C(t)$$

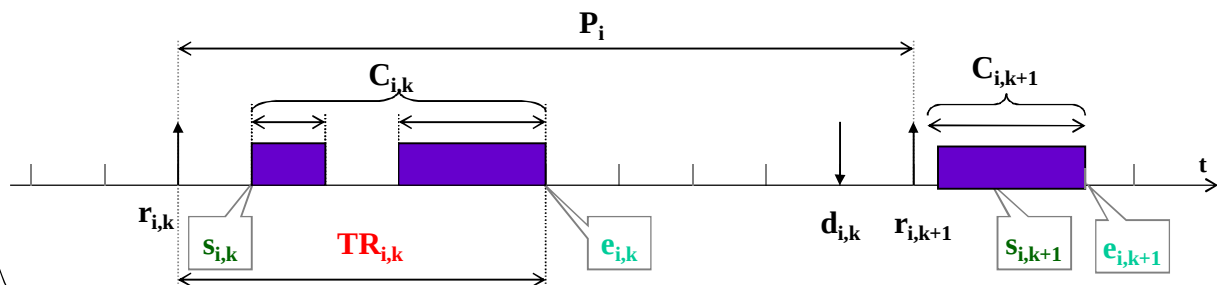


Paramètres dynamiques de tâche (2/4)



Paramètres dynamiques de tâche (3/4)

- Début d'exécution de la $k^{\text{ième}}$ instance de la tâche T_i : $s_{i,k}$
- Fin d'exécution de la $k^{\text{ième}}$ instance de la tâche T_i : $e_{i,k}$
- Temps de réponse de la $k^{\text{ième}}$ instance de la tâche T_i : $TR_{i,k} = e_{i,k} - r_{i,k}$
 - Temps de réponse maximum de la tâche T_i : $TR_i = \max_k\{TR_{i,k}\}$
 - Temps de réponse minimum de la tâche T_i : $TR_{i,\min} = \min_k\{TR_{i,k}\}$
 - Temps de réponse moyen de la tâche T_i : $TR_{i,\text{moy}} = \sum_k\{TR_{i,k}\}/k$



Paramètres dynamiques de tâche (4/4)

→ **Gigue absolue** max d'activation de T_i :

$$GA_i = \max_k(s_{i,k} - r_{i,k}) - \min_k(s_{i,k} - r_{i,k})$$

→ **Gigue absolue** max de terminaison de T_i :

$$GT_i = \max_k(e_{i,k} - r_{i,k}) - \min_k(e_{i,k} - r_{i,k})$$

Validation d'applications TR critiques

- Identifier à partir des spécifications la classe de problèmes à résoudre
- Déterminer les scénarii pire cas pour ce problème et exprimer les conditions de faisabilité (CF) associées
- Donner les valeurs numériques des paramètres des CF
- Vérifier que les CF sont validées

Problème : Il y a très peu de conditions de faisabilité pour répondre aux besoins réels.
⇒ Recherche active pour trouver des CF.

2. Classes d'algorithmes d'ordonnancement

- **Ordonnancement** = liste (partiellement) ordonnée d'accès aux ressources partagées
- **Entités ordonnançables** : tâches, threads, processus, messages, ...
- **Ordonnancement valide** : Ordonnancement qui satisfait les CT

Algorithmes d'ordonnancement classiques

● FIFO (à priorités dynamiques)

- Les tâches sont traitées dans l'ordre de leur demande d'activation.
- Toutes les tâches ont la même importance pour FIFO.
- FIFO minimise le temps de réponse de l'ensemble des tâches.
- FIFO n'est pas optimal.

● Round Robin (à priorités dynamiques)

- Le processeur est attribué à tour de rôle aux tâches actives.
- RR est équitable.
- RR n'est pas optimal pour des tâches temps réel.

FIFO et RR ne garantissent pas les contraintes temporelles dans le cas général

Condition suffisante pour FIFO : $\text{somme}(C_i) \leq \min(P_i)$

Critères de classification des algorithmes (1/3)

Ordonnancement statique

- Connaissance statique de tous les paramètres de tâches
- Adapté à des systèmes « figés »

Beaucoup de résultats pour l'analyse de l'ordonnançabilité

Prédictibilité : Oui

Ordonnancement dynamique

- Connaissance des paramètres de tâches au moment de leur arrivée
- Tient compte des changements ou événements

Peu de résultats pour l'analyse de l'ordonnançabilité

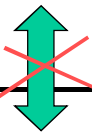
Prédictibilité : NON

Ordonnancement hors ligne

- Calcul préalable d'une séquence d'ordonnancement suivie en ligne par un dispatcheur

Ordonnancement en ligne

- Elaboration de séquence d'ordonnancement à l'exécution



Ordonnancement statique peut être fait en ligne
Ordonnancement dynamique ne peut pas être fait hors ligne

Statique = prédictible
Hors ligne = ordre défini à l'avance

Critères de classification des algorithmes (2/3)

Préemptif

Non préemptif

Non ordonnançable en préemptif



Non ordonnançable en non préemptif

Dirigé par une table statique

Dirigé par les priorités

A priorités statiques (FP)

A priorités dynamiques (DP)

S'il trouve toujours une solution
faisable lorsqu'il en existe une.

Optimal (dans sa classe FP ou DP)

Non optimal

Si on diminue la demande
processeur, l'application reste
ordonnançable.

Stable

Instable

Pour un algo optimal, toutes les solutions sont équivalentes

Critères de classification des algorithmes (3/3)

Monoprocasseur

Multiprocasseur

Local

Global (distribué)

Oisif

Retarde des décisions même si la file n'est pas vide (utile en milieu décentralisé)

Non oisif

Pas de **temps creux** si une tâche est prête

Oisif pour absorber la gigue

Autres critères de classification des algorithmes (1/2)

- **Complexité**
- **Tolérance aux fautes**
- **Critère optimisé**
 - **Nombre de tâches ne respectant pas les CT**
 - **Longueur de l'ordonnancement**
 - **Equilibrage de charge entre les processeurs**
 - **Charge de communication**
 - **Erreur accumulée** (tâches incrémentales/résultat imprécis)
 - **Autre**

Autres critères de classification des algorithmes (2/2)

- **Technique utilisée**

- ➔ **Théorie des graphes**
- ➔ **Recuit simulé**
- ➔ **Algorithmes génétiques**
- ➔ **Réseaux de neurones**
- ➔ **Théorie des files d'attente**
- ➔ **Logique floue**
- ➔ **Autre**

Rappels sur la complexité des algorithmes

→ **Algorithme en $O(f(n))$:**

- * $O(\log(n))$: logarithmique
- * $O(n)$: linéaire
- * $O(n^k)$: polynomial
- * $O(k^n)$: exponentiel

→ **Problème polynomial (P)** : problème pouvant être résolu en un temps polynomial

→ **Classe P** : ensemble des problèmes P

→ **Problème non polynomial (NP)** : problème ne pouvant pas être résolu en un temps polynomial

→ **Classe NP** : ensemble des problèmes NP

→ **Problème NP-complet ('NP-complete')** : un problème p appartenant à NP est NP-complet si tout autre problème de NP peut être transformé en p en un temps polynomial.

→ **Problème NP-difficile ('NP-Hard')** : un problème p est NP-difficile si tout problème appartenant à NP se transforme en p en un temps polynomial, mais on ne sait pas démontrer si p appartient à NP.

Quelques propriétés des problèmes

- **Problème faisable** : Existence d'un ordonnancement valide
- **Condition de faisabilité** : $CF(P) \Rightarrow P$ ordonnançable
- **Complexité du problème** : polynomial (P), NP-complet, NP-difficile
- **Complexité des solutions** :
 - ➔ Pour certains problèmes polynomiaux : on connaît des solutions optimales de complexité polynomiale (**utilisables en ligne**)
 - ➔ Pour certains problèmes non polynomiaux : on connaît des solutions optimales de complexité exponentielle (**utilisables en hors ligne**)
 - ➔ Pour les autres problèmes non polynomiaux : on ne connaît que des solutions sous-optimales de complexité polynomiale

Récapitulatif

- **Modèle de tâches** : défini par les contraintes des tâches
- **Pour chaque modèle de tâche, on a un ou plusieurs algorithmes d'ordonnancement.**
- **Mise en œuvre d'une application sur une machine cible**
A faire HORS LIGNE :
 - ➔ **Vérifier que le problème est faisable (indépendamment de tout algorithme d'ordonnancement).**
 - ➔ **Choisir un algorithme d'ordonnancement.**
 - ➔ **Vérifier l'ordonnançabilité en utilisant les propriétés de l'algorithme choisi.**
 - ➔ **Si l'application est ordonnançable, alors l'implanter.**

Récapitulatif (suite)

- **Tout algorithme d'ordonnancement de tâche TR**

- ➔ S'appuie/utilise un ordonnanceur de bas niveau (ou dispatcheur) qui s'exécute en ligne en même temps que les tâches d'application.
- ➔ Fait certains calculs hors ligne (eg. Affectation des priorités statiques)
- ➔ Effectue certains calculs en ligne (eg. Tri des files d'attente, calcul de la laxité...).

Attention : si la vérification de l'ordonnançabilité n'a pas été faite a priori, des fautes temporelles peuvent surgir en ligne et l'ordonnanceur n'est pas censé les détecter. L'ordonnanceur peut continuer à allouer le processeur à des tâches alors que leurs échéances sont ratées.